INFRA · PART 28

Run a 70B Model on Your Laptop.

No API key, no bill, no data leaving the machine. Here is exactly what fits in your RAM, how fast it will run, and the one command that starts it.

`Ollama` `llama.cpp` `GGUF` `Q4_K_M` `MLX`

16 slides · the stack · quantization · VRAM math · what fits where · speed · offline RAG

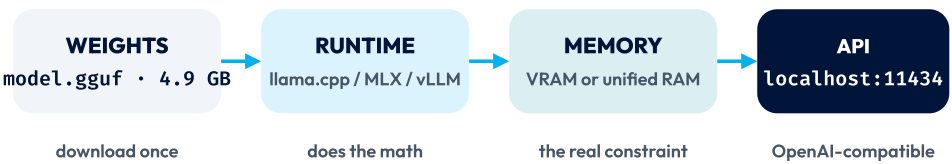
AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

What "running locally" actually means

A model is a file of weights. A runtime loads it into memory and turns your prompt into tokens, one forward pass at a time. Everything happens on your silicon.



Your app talks to localhost exactly as it talks to OpenAI. Nothing else in the code changes.

Privacy

Medical notes, legal contracts, source code. Some data cannot leave the building. Local is the only compliant option.

Cost

A dev running 5K requests a day against an API spends ≈$50–300 a month. A laptop model costs electricity.

Offline & latency

Works on a plane, in a factory, on a ship. First token in ≈100 ms with no network round-trip.

Learning

You see the KV cache grow, watch tokens/s drop with context, feel quantization. APIs hide all of it.

Five runtimes, one decision tree

Most of them wrap the same two engines. Pick by what hardware you have and whether you want a GUI, a CLI, or a server.

TOOL	WHAT IT IS	BEST ON	PICK WHEN
llama.cpp	C++ engine, GGUF format, CPU + GPU offload	Anything	You want control and every flag
Ollama	llama.cpp wrapped in a model manager + REST API	Mac, Linux, Windows	You want `ollama run` and an API in 2 min
LM Studio	Desktop GUI over llama.cpp and MLX	Mac, Windows	You want to browse and chat, no terminal
MLX	Apple's array framework, native Metal	Apple silicon	Max speed on M-series, fine-tuning on Mac
vLLM	Server engine with PagedAttention, batching	NVIDIA GPUs	Many concurrent users, not a laptop

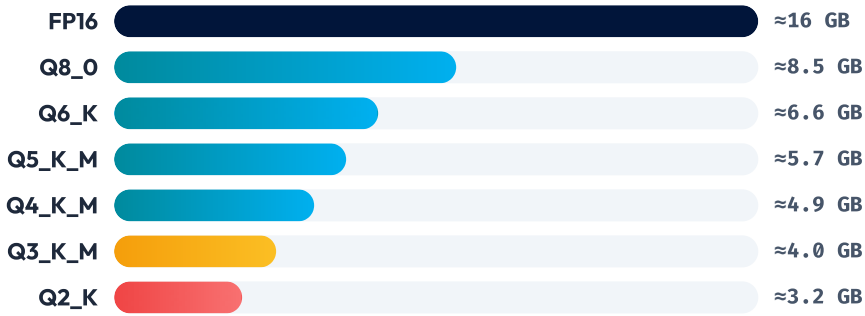


>_ Start with Ollama. When you hit a wall (speed, a flag it hides, a format it lacks) drop down to llama.cpp or MLX. Same models, same GGUF files.

GGUF and the Q-number

GGUF is the file format llama.cpp reads. The Q-suffix is how many bits each weight was squeezed into. Fewer bits, smaller file, slightly dumber model.

SAME 8B MODEL, FILE SIZE BY QUANTIZATION



LEVEL	QUALITY VS FP16	USE
Q8_0	Practically identical	You have the memory and want a reference
Q5_K_M / Q6_K	≈ no visible loss	Best quality that still fits comfortably
Q4_K_M	Small loss, rarely noticed	The default. Best size-to-quality trade
Q3 and below	Noticeable degradation	Only to squeeze a bigger model into small RAM

A bigger model at Q4 almost always beats a smaller model at Q8. Spend your memory on parameters, not on precision.

Will it fit? Do the arithmetic

One formula answers "can my machine run this" before you download 40 GB to find out.

Memory needed

```
params × bytes per weight  +  KV cache  +  ≈1 GB runtime overhead
```

Bytes per weight

FP16 = 2.0 · Q8 ≈ 1.06 · Q6 ≈ 0.8 Q5_K_M ≈ 0.7 · Q4_K_M ≈ 0.6 · Q3 ≈ 0.5

KV cache

Grows with context length. For an 8B model at FP16 ≈ 0.5 MB per token → 8K ctx ≈ 4 GB, 32K ≈ 16 GB. Q8 KV halves it.

TWO WORKED EXAMPLES

8B · Q4_K_M · 8K context

```
8.0 × 0.6 = 4.8 GB weights
8K × 0.5MB = 4.0 GB KV (fp16)
+ 1.0 GB overhead
```

≈ 10 GB → fits 16 GB Mac

70B · Q4_K_M · 8K context

```
70 × 0.6 = 42 GB weights
8K ctx ≈ 2.5 GB KV (GQA)
+ 1.0 GB overhead
```

≈ 46 GB → needs 64 GB unified

On a Mac, macOS keeps ≈25% of unified memory for itself by default. A 16 GB machine offers ≈12 GB to the model. Budget for that.

What fits on your machine

Comfortable means the model, an 8K context and the OS all coexist without swapping. Squeeze means it runs, slowly, with a short context.

MEMORY	TYPICAL MACHINE	COMFORTABLE	SQUEEZE
8 GB	Base laptop, RTX 3050/4060	1–4B at Q4–Q8 (Gemma 3 4B, Qwen 3 4B, Phi-4-mini)	7–8B at Q3, 2K ctx
16 GB	M1/M2/M3 base Mac, RTX 4080	7–9B at Q4–Q6 (Llama 3.1 8B, Qwen 3 8B, Gemma 3 12B Q4)	14B at Q4, short ctx
24 GB	RTX 3090/4090, M-series 24 GB	12–14B at Q6, 8B at Q8 + 32K ctx	27–32B at Q4 (Gemma 3 27B, Qwen 3 32B)
32 GB	M-series 32 GB	27–32B at Q4–Q5	70B at Q2, barely
64 GB	M-Max / M-Ultra 64 GB	70B at Q4 (Llama 3.3 70B), 32B at Q8	≈110B MoE at Q4
128 GB+	M-Ultra 128–192 GB	70B at Q8, 120B-class MoE at Q4	≈235B MoE (Qwen 3) at Q3–Q4

Why Macs punch above their weight

Unified memory: the GPU sees all 64 or 128 GB. A discrete NVIDIA card is capped at its own 8–24 GB VRAM, then spills to slow system RAM.

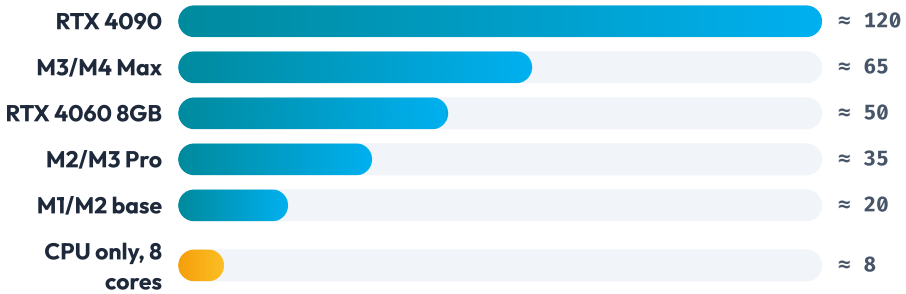
Why NVIDIA is still faster

A 4090 has ≈1 TB/s memory bandwidth vs ≈400 GB/s on an M3 Max. When the model fits in VRAM, the GPU wins on tokens/s by 2–3×.

Tokens per second: what to expect

Generation is memory-bandwidth bound. Every token requires reading all the weights once, so speed ≈ bandwidth ÷ model size.

8B MODEL AT Q4_K_M · GENERATION TOKENS/S · APPROXIMATE



≈ 5–7
tok/s feels like reading speed

≈ 20+
tok/s feels instant for chat

÷ 9
70B is ≈9× slower than 8B on the same box

Prompt processing is different
Reading a 10K-token prompt is compute bound. ≈500–2,000 tok/s on GPU, ≈50–100 on CPU. Long RAG contexts feel slow on CPU-only boxes.

Speed drops with context
Each new token attends to all previous ones. At 32K context an 8B model can lose 30–50% of its tokens/s. Flash attention helps.

Good local models to start with

Open-weight quality moved fast. An 8B at Q4 is roughly GPT-3.5 class. 30B-class models handle most coding and RAG work.

FAMILY	SIZES	GOOD AT	NOTE
Llama 3.x / 4	1B, 3B, 8B, 70B, MoE	General chat, tool calling, long context	Largest ecosystem of fine-tunes
Qwen 3	0.6B–32B dense, 30B/235B MoE	Coding, math, multilingual, reasoning toggle	Strong at every size
Gemma 3	1B, 4B, 12B, 27B	Vision + text, 140 languages, 128K ctx	27B ≈ much larger models
Phi-4 / mini	3.8B, 14B	Reasoning, math, small-footprint	Weak on trivia, strong on logic
Mistral Small	24B	Fast, instruction following, function calls	Apache 2.0
DeepSeek-R1 distills	1.5B–70B	Step-by-step reasoning on Qwen/Llama bases	Slow: thinks before answering
Nomic / BGE / E5	≈100–600M	Embeddings for local RAG	Run alongside the LLM

Test on *your* task, not a leaderboard. A 4B model that nails your extraction schema beats a 70B that is slow and generic.

Local in one environment variable

Ollama serves an OpenAI-compatible endpoint. Point the SDK's base URL at it and your existing code runs against the local model unchanged.

```
# terminal
$ ollama pull qwen3:8b && ollama serve   # ≈5 GB once · REST on :11434
# app.py · same code for OpenAI, Ollama, vLLM, LM Studio
import os
from openai import OpenAI

client = OpenAI(
    base_url=os.getenv("LLM_BASE_URL", "https://api.openai.com/v1"),
    api_key=os.getenv("LLM_API_KEY", "ollama"),   # any string works loc
)
resp = client.chat.completions.create(
    model=os.getenv("LLM_MODEL", "gpt-4.1-mini"),
    messages=[{"role": "user", "content": "Summarise this contract…"}],
    temperature=0.2, stream=True,
)
for chunk in resp:
    print(chunk.choices[0].delta.content or "", end="")

# switch to local:
# LLM_BASE_URL=http://localhost:11434/v1 LLM_MODEL=qwen3:8b python app.p
```

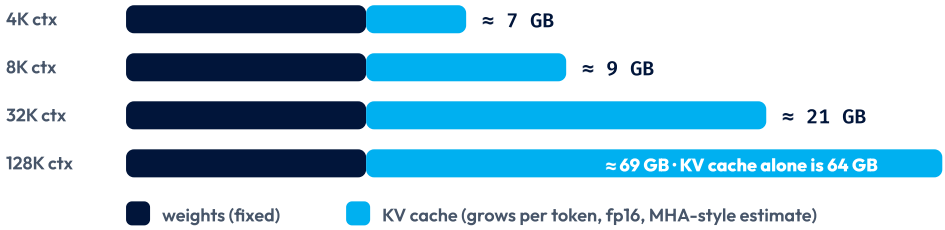
Tool calling and JSON mode work for models that support them. `num_ctx` defaults to 4K in Ollama, raise it for RAG.

TRADE-OFF

Context length eats your memory

The model card says 128K context. Your RAM says otherwise. The KV cache is the hidden second model you are loading.

8B MODEL, Q4 WEIGHTS (4.8 GB) · TOTAL MEMORY BY CONTEXT LENGTH



Models with GQA (Llama 3, Qwen 3) use ≈4–8× less KV. Still the dominant cost past 32K.

Quantize the KV cache

llama.cpp flag `-ctk q8_0 -ctv q8_0` (Ollama: `OLLAMA_KV_CACHE_TYPE=q8_0`) halves KV memory with negligible quality loss.

Set the context you need

RAG with 5 chunks of 500 tokens plus a 1K prompt needs ≈4K. Do not allocate 32K for a 4K job, it steals memory that could hold a bigger model.

OFFLINE RAG

A RAG pipeline with zero network calls

Embeddings, vector store, reranker and generator all run on one laptop. This is the setup for legal, medical and air-gapped work.



Memory on a 16 GB Mac: embed ≈0.3 GB · reranker ≈0.6 GB · LLM Q4 + 8K ctx ≈ 9 GB · vector index for 50K chunks ≈ 0.2 GB
Total ≈ 10 GB. Fits, with room for the browser you will keep open.

```
# Ollama serves embeddings too
$ ollama pull nomic-embed-text
import ollama
vec = ollama.embed(model="nomic-embed-text", input="return policy is 14
print(len(vec))   # 768 dims
```

⚠ The local trap

Small models follow "answer only from context" less reliably. Keep chunks short, put the instruction last, and validate citations in code.

✓ Where local RAG shines

Personal notes, a codebase, one company's policy docs. Bounded corpus, private data, no per-query cost.

Six flags that change everything

Defaults are conservative. These settings turn a slow local model into a fast, useful one.

KNOB	LLAMA.CPP / OLLAMA	WHAT IT DOES
GPU layers	`-ngl 99 · auto in Ollama`	Layers on GPU. Partial offload (20 of 32) still beats CPU ≈3×.
Context	`-c 8192 · num_ctx`	Window size. Sets KV allocation. Bigger = more memory, slower.
Flash attention	`-fa · OLLAMA_FLASH_ATTENTION=1`	Faster, less memory. Required for KV quantization.
KV type	`-ctk q8_0 -ctv q8_0`	Quantize the cache. Halves KV memory. Avoid q4_0.
Threads	`-t 8 · num_thread`	CPU threads. Set to physical cores, not logical.
Batch	`-b 512 · num_batch`	Prompt tokens per step. Bigger = faster ingestion, more memory.

```
# Ollama Modelfile: bake settings into a named model
FROM qwen3:8b
PARAMETER num_ctx 16384
PARAMETER temperature 0.2
SYSTEM "Answer only from the provided context. Cite chunk ids."   # $ ol
```

When local is the wrong call

Local is a tool, not a religion. There are five situations where an API is cheaper, better, or both.

✕ Go API

- Frontier reasoning: a 70B local model is still well behind the best hosted models on hard tasks
- Many concurrent users: one laptop serves one user well
- Bursty traffic: idle GPU is wasted money
- 200K+ context regularly
- Team without anyone to babysit GPU drivers

✓ Go local

- Data cannot leave the machine or country
- Steady, high volume of simple tasks (classification, extraction, summaries)
- Offline or edge deployment
- Development and eval loops: free iterations
- Learning how inference really works

BREAK-EVEN, ROUGHLY

API · small hosted model

`≈ $0.15–0.60 per 1M tokens`

1M tokens/day ≈ $5–20/month. Hard to beat.

Local · 8B on a $2K Mac

`≈ 20M tokens/day max at 65 tok/s × 4 h`

Pays back only if you run it hard, or privacy forces it.

Common pattern: local for dev, evals and private data; hosted frontier model for the 10% of requests that need it. A router decides per request.

TROUBLESHOOTING

Six things that go wrong the first week

Every one of these has a one-line fix once you know the cause.

01 **Painfully slow, fans screaming.** Model spilled from GPU to system RAM. Pick a smaller quant or lower context so it fits entirely in VRAM/unified memory.

02 **Answers cut off mid-sentence.** Context window too small for prompt + output. Raise `num_ctx`, or shrink the RAG context.

03 **Model ignores the system prompt.** Wrong chat template for that model. Use the official Ollama tag or check the GGUF has the template baked in.

04 **Gibberish or repeated words.** Q2/Q3 quant of a small model, or a broken GGUF conversion. Go up one quant level or re-download.

05 **First request takes 30 s.** Cold load from disk. Ollama keeps models warm for 5 min; set `OLLAMA_KEEP_ALIVE=-1` to pin it.

06 **Tool calls come back as plain text.** Model or tag lacks tool support. Use a "-tools" or instruct variant, or parse JSON from the text yourself.

`>_` `ollama ps` shows what is loaded and how much is on GPU vs CPU. If it says 60%/40%, that split is your slowness.



Eight rules for running local

Follow these and you skip the week of trial and error most people go through.

1. Do the **memory math** before the download.
2. **Q4_K_M** by default. Bigger model at Q4 beats smaller at Q8.
3. Fit the whole model in **GPU / unified memory**. Spilling is the #1 slowness.
4. Set **context to what the task needs**, not the maximum.
5. Code against the **OpenAI-compatible API**. Swap providers with an env var.
6. Turn on **flash attention** and q8 KV cache.
7. Evaluate on **your task**, not a leaderboard.
8. Route: local for private and cheap, **hosted for hard**.

NUMBERS TO REMEMBER

0.6	**≈ 5 GB**	**≈ 42 GB**	**20 tok/s**
bytes per weight at Q4_K_M	8B at Q4, weights only	70B at Q4, weights only	the "feels instant" line

Local LLMs on one slide

CONCEPT	REMEMBER
Stack	Ollama to start. llama.cpp for control, MLX on Mac, vLLM for serving on NVIDIA.
GGUF	The file format. Q-suffix = bits per weight. Q4_K_M is the default.
Memory	params × bytes/weight + KV cache + 1 GB. Q4 ≈ 0.6 B/weight, FP16 = 2.
8B Q4	≈ 5 GB weights, ≈ 9–10 GB with 8K context. Fits a 16 GB machine.
70B Q4	≈ 42 GB weights, ≈ 46 GB total. Needs 64 GB unified memory.
Fits where	8 GB → 4B · 16 GB → 8B · 24 GB → 14B, 32B squeeze · 64 GB → 70B.
Speed	Bandwidth ÷ model size. 8B Q4: ≈120 tok/s 4090, ≈65 M-Max, ≈20 M1 base.
Context	KV cache grows per token. Quantize it (q8_0), size it to the task.
Models	Qwen 3, Llama 3.x/4, Gemma 3, Phi-4, Mistral Small, R1 distills, nomic-embed.
Code	OpenAI SDK + base_url=localhost:11434/v1. One env var to switch.
Offline RAG	docling → nomic-embed → Chroma/LanceDB → bge-reranker → local LLM. ≈10 GB.
Not local	Frontier reasoning, many users, bursty load, 200K+ context.

A model is a file. Fit it in memory, feed it tokens, and the API bill becomes optional.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost